RAG-Based E-commerce Customer Support Chatbot

NLP Final Project Report

Author: Abdelrahman Ihab

Repository: https://github.com/Abdelrahman-288/Rag-Ecommerce-Customer-Support-Chatbot

Academic Year 2026

Contents

1 Introduction

1.1 Project Overview

This project implements an end-to-end, retrieval-augmented generation (RAG) chatbot designed to handle e-commerce customer support queries. Rather than treating support automation as a single classification or generation task, the system decomposes each incoming customer message into four sequential NLP stages before producing a final response: **language detection**, **sentiment analysis**, **intent classification**, and **semantic retrieval**, followed by a **grounded LLM generation** step.

The motivating idea is that a real customer support system cannot treat every message the same way. A greeting needs no computation beyond a canned reply; a plain order-status question needs to be grounded in real support documentation; and a message from a visibly frustrated customer needs to be prioritized and escalated, even when the specific request is ambiguous. This project's central design contribution is a **multi-signal, confidence-aware routing layer** that makes exactly these distinctions.

1.2 Objectives

- Build independent, well-evaluated NLP components for language detection, sentiment classification, and intent classification.
- Build a retrieval-augmented generation pipeline grounded in a real customer-support knowledge base.
- Integrate all components into a single orchestrated pipeline with principled, testable routing logic.
- Expose the system through two independent interfaces (a chat UI and a REST API) that share identical underlying logic.
- Validate the system through both automated testing and live, manual adversarial testing, and document every real limitation discovered.

1.3 Report Structure

This report follows the same 12-stage structure used to build the project: environment setup, version control, the three independent classifiers, the RAG pipeline, pipeline integration, the two user-facing interfaces, automated testing, and final documentation. Each section explains *what* was built, *why* that design was chosen over alternatives, and, where relevant, *what went wrong and how it was fixed* – several real bugs were found only through live testing after initial implementation, and are documented here as part of the engineering process rather than omitted.

2 System Architecture

2.1 High-Level Pipeline

Every customer message flows through the same fixed pipeline, illustrated in Figure 1.

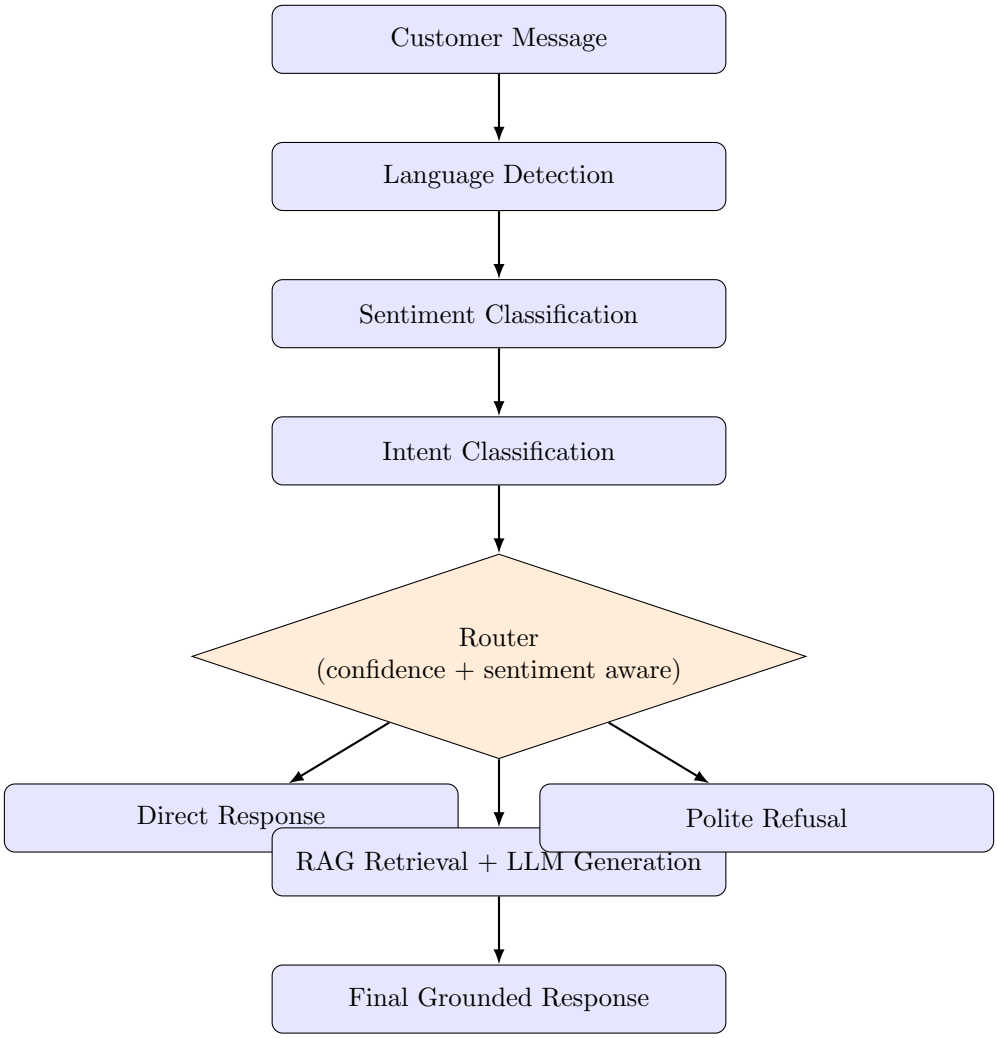

Figure 1: End-to-end chatbot pipeline architecture.

2.2 Routing Logic

The router is the component that most distinguishes this implementation from a simple RAG chatbot. Its rules, in order of precedence, are:

1. **Small talk** (`greeting`, `goodbye`, `gratitude`), detected by a closed-vocabulary rule layer, receives an instant canned response. No retrieval or LLM call occurs, avoiding unnecessary latency and API cost.

2. **Complaint** intent is *always* routed to RAG generation plus human escalation, regardless of classifier confidence. This reflects the project requirement that complaints receive priority handling unconditionally.

3. **Support intents** (`order_status`, `order_management`, `billing_and_refunds`, `account_management`) predicted above a confidence threshold are routed to RAG retrieval and grounded generation.

4. **Low-confidence predictions** are not blindly trusted. If confidence falls below the routing threshold (0.55) but sentiment is independently detected as *Negative/Frustrated*, and the confidence is still above a noise floor (0.30), the message is treated as an unclear complaint and escalated – a second, independent signal is used to avoid giving a cold refusal to a genuinely upset customer whose specific request the intent model could not classify confidently.

5. Predictions below the noise floor are treated as unreliable regardless of sentiment (protecting against, for example, gibberish input that is spuriously classified as negative) and receive a polite refusal.

This routing logic was not designed correctly on the first attempt – Section 6 documents how two real bugs in this exact logic were found through live testing and fixed.

2.3 Interfaces

Two interfaces are provided, both calling the identical `process_message()` function in `src/chatbot/pipeline.` ensuring zero duplicated business logic:

- **Streamlit** – an interactive chat UI with a pipeline-details debug panel, FAQ quick-action buttons, dark mode, an animated typing indicator, and a thumbs up/down feedback logging mechanism.
- **FastAPI** – a `POST /chat` REST endpoint returning the same structured JSON response, suitable for integration into an external system.

3 Technologies Used

Category	Technology
Language	Python 3.11
Deep learning	PyTorch 2.14 (CPU-only build)
Transformer models	Hugging Face Transformers (DistilBERT)
Embeddings	Sentence-Transformers (`all-MiniLM-L6-v2`)
Classical ML	scikit-learn (TF-IDF, Logistic Regression)
Vector search	FAISS
LLM generation	Groq API (`gpt-oss-120b`)
Web UI	Streamlit
REST API	FastAPI + Uvicorn
Testing	Pytest, Streamlit `AppTest`
Data source	Hugging Face `datasets`
Version control	Git + GitHub

Table 1: Core technology stack.

3.1 A Note on the CPU-Only Constraint

A deliberate design constraint of this project was to avoid installing CUDA-enabled PyTorch, keeping the local development install footprint small. `src/utils/device.py` centralizes device selection and would automatically use a GPU if a CUDA-enabled PyTorch build were installed instead, but all training and inference in this project was performed on CPU. This directly informed several architecture decisions – most notably, favoring `TF-IDF + Logistic Regression` over deep learning for language detection and intent classification (fast, effective, and appropriate given the strength of classical methods on these particular tasks), while still using a fine-tuned `DistilBERT` transformer for sentiment classification, accepting a longer (approximately two-hour) CPU training time in exchange for stronger contextual understanding on a genuinely harder task.

4 Datasets

Dataset	Used for	S
`papluca/language-identification`	Language detection	9
`dair-ai/emotion`	Sentiment classification	2
`bitext/Bitext-customer-support-llm-chatbot-training-dataset`	Intent classification & RAG KB	2

Table 2: Datasets used across the project.

A recurring theme across this project is **domain mismatch** between publicly available training datasets and the actual deployment distribution (short, transactional customer-support messages). This mismatch was identified and corrected in two of the three trained models, as detailed in Sections 5.3 and 5.4.

5 Implementation: Stage by Stage

5.1 Stage 1 – Environment and Project Setup

A clean Python virtual environment was established with a professional, modular folder structure separating core logic (`src/`), scripts, the two interfaces (`app/`, `api/`), configuration, models, data, and tests. A centralized `get_device()` utility abstracts CPU/GPU selection so that no other module needs to know which device is in use.

5.2 Stage 2 – Version Control

The project was initialized with Git and pushed to a public GitHub repository. Notably, during the first push, GitHub's *secret scanning push protection* correctly blocked a commit containing a real Groq API key accidentally placed in `.env.example` instead of `.env`. The key was rotated immediately, the file corrected, and the commit amended before a successful push – an example of a real security safeguard functioning as intended during development.

5.3 Stage 3 – Language Detection

5.3.1 Approach

A `TF-IDF (character n-grams, 1--3) + Logistic Regression` classifier was trained across all 20 languages in the dataset. Character-level n-grams were chosen over word-level features because language identification depends on letter-pattern statistics (e.g. diacritics, digraphs) rather than vocabulary, and this approach generalizes better to short or unseen words.

5.3.2 A Real Problem: Short-Text Domain Shift

Initial evaluation on the held-out test set showed 99.5% accuracy. However, manual testing with realistic short customer queries (e.g. *"Where is my order?"*) revealed significantly degraded confidence and, in some cases, incorrect predictions – because the source dataset consists of full sentences, while real customer messages are short. This was corrected by **augmenting the training data with short word-span snippets** sampled from the original sentences, with a bias toward preserving sentence-initial characters (important for cues such as the Spanish inverted question mark "¡"). This is a clear example of a train/deployment distribution mismatch identified and corrected through testing rather than benchmark performance alone.

5.3.3 Final Results

Metric	Value
Test Accuracy	99.53%
Macro F1	0.9953

Residual, documented weaknesses remain between Spanish/Portuguese (the two most character-similar languages in the set) and between romanized Hindi/Urdu and Swahili.

5.4 Stage 4 – Sentiment Classification

5.4.1 Approach

A fine-tuned **DistilBERT** model classifies messages into three routing-relevant buckets: *Negative/Frustrated*, *Neutral*, and *Positive/Satisfied*. The source dataset's six fine-grained emotions were mapped as follows:

Emotion(s)	Mapped Sentiment
sadness, anger, fear	Negative/Frustrated
joy, love	Positive/Satisfied
surprise	Neutral (documented approximation)

DistilBERT was selected over a from-scratch RNN/LSTM as the modern, industry-standard approach for text classification via transfer learning, and over full BERT for its favorable speed/accuracy trade-off on CPU.

5.4.2 Problem 1: Class Imbalance

After mapping, *Neutral* represented only 3.6% of training examples. Standard cross-entropy training would allow the model to largely ignore this minority class while maintaining high overall accuracy. This was addressed with **class-weighted loss** (inverse class frequency), implemented via a custom `Trainer` subclass, since Hugging Face's `Trainer` does not natively support weighted loss.

5.4.3 Problem 2: Domain Shift on Support Queries

Manual testing revealed that plain transactional questions (*"Where is my package?"*) were confidently misclassified as *Negative/Frustrated*, since the training data (Twitter posts about personal feelings) contains no analogous neutral, transactional phrasing. This was corrected by adding a small set of hand-written, naturally-phrased support examples to the **training split only**, preserving the integrity of test-set evaluation. This improved real-world calibration at a small, deliberate cost to held-out test macro F1 (0.913 → 0.890), a trade-off favoring deployment-relevant performance over benchmark performance.

5.4.4 Problem 3: Negation Handling

The most subtle bug found in the entire project: negated positive statements such as *"this is **not** acceptable"* were misclassified as *Positive/Satisfied* with over 99% confidence, apparently because the model keyed on the positive word "acceptable" while under-weighting the negation. This was fixed by adding hand-written **negated-positive** *and* **negated-negative** examples to training data – the latter specifically to avoid overcorrecting into a blanket "negation always means negative" heuristic.

5.4.5 Final Results (after all fixes)

Metric	Value
Test Accuracy	97.70%
Macro F1	0.912
Neutral-class Recall	0.86 (up from 0.73 pre-fix)

5.5 Stage 5 – Intent Classification

A `TF-IDF + Logistic Regression` classifier routes messages into six categories: `account_management`, `billing_and_refunds`, `complaint`, `order_management`, `order_status`, and `out_of_scope`. *Greeting*, *goodbye*, and *gratitude* are handled by a separate, deterministic rule-based layer instead of the ML classifier, since the source dataset contains zero training examples for these categories

– precision-first regular expressions catch the closed vocabulary of small talk, with any miss safely falling through to the ML classifier.

Metric	Value
Test Accuracy	99.78%
Macro F1	0.9974
Misclassifications	8 out of 3,642 test examples

The eight residual errors occur at genuine category boundaries introduced by condensing the source dataset's 27 fine-grained intents into 6 routing categories (e.g. `track_refund` confused with `order_status`), rather than representing a model deficiency.

5.6 Stage 6 – Retrieval-Augmented Generation

5.6.1 Pipeline

1. **Knowledge base**: the Bitext dataset's instruction/response pairs (26,872 entries) form the retrieval corpus.
2. **Embeddings**: `sentence-transformers/all-MiniLM-L6-v2` embeds both the knowledge base and incoming queries.
3. **Vector store**: a FAISS index (24,274 vectors after knowledge-base construction) supports fast cosine-similarity search.
4. **Generation**: the Groq API (`gpt-oss-120b`) generates a response grounded strictly in the top-k retrieved documents, with an explicit system prompt instructing the model to avoid inventing facts, to never repeat template placeholders verbatim to the customer, and to honestly state when retrieved context is insufficient rather than guessing.

5.6.2 Qualitative Evaluation

The four evaluation queries suggested in the original project brief were tested manually, all four retrieving topically correct top-3 results with similarity scores between 0.69 and 0.99, confirming the embedding and retrieval pipeline generalizes correctly beyond exact-phrase matches (e.g. correctly ignoring incidental profanity in a query while retrieving the correct delivery-tracking intent).

5.7 Stage 7 – Chatbot Pipeline Integration

The four independent components were unified in `src/chatbot/pipeline.py`, with routing logic in a separate `router.py` module and a typed `ChatbotResponse` dataclass in `schemas.py` shared by both interfaces.

5.8 Stage 8 – Streamlit Interface

The chat interface was iteratively refined based on live feedback, evolving from a functional debug-oriented layout into a polished, AWS-Assistant-inspired design featuring:

- A gradient header and FAQ quick-action buttons for common queries.
- A togglable dark mode implemented via CSS overrides (Streamlit's native theme system only applies at startup and cannot be toggled at runtime).

- An animated three-dot typing indicator, rendered via `st.empty()` placeholders that Streamlit updates progressively during pipeline execution.
- A thumbs up / thumbs down feedback mechanism, logging each vote alongside full pipeline context (language, sentiment, intent, route) to a CSV file for later analysis.

5.9 Stage 9 – Automated Testing

An automated test suite was built using Pytest, mocking the external Groq API call so that tests run in seconds with no network dependency, per the project's testing requirements. The suite grew across the project as new behaviors and bugs were discovered:

Test file	Test count
`test_language_detection.py`	5
`test_sentiment.py`	7
`test_intent.py`	8
`test_retriever.py`	5
`test_pipeline.py`	10
`test_multilingual.py`	27
Total	**62**

Notably, several tests exist specifically as *regression tests* for bugs found during manual testing (e.g. `test_pipeline_low_confidence_but_frustrated_routes_to_escalation`), permanently locking in fixes so they cannot silently regress.

5.10 Stage 10 – FastAPI Interface

A `POST /chat` endpoint exposes the identical pipeline via a typed Pydantic request/response model, alongside a `GET /` health check. Live testing of this endpoint directly led to the discovery of the router bug described in Section 6.

5.11 Stage 11 – Documentation

A comprehensive `README.md` was written covering architecture, installation, training, evaluation results, and – deliberately extensively – an honest **Limitations** section, since a system that documents its own boundaries accurately is considerably more trustworthy than one claiming unqualified success.

5.12 Stage 12 – Final Verification and Submission

The final stage consisted of a full verification pass: confirming a clean git history, confirming no secrets were ever committed (verified via `git log --all --full-history -- .env`), confirming the complete automated test suite passes, and a final manual multilingual adversarial testing pass, described in Section 7.

6 Real Bugs Found and Fixed

A defining characteristic of this project's development process was that every major fix originated from **live, manual testing** rather than being anticipated in advance. This section catalogs each one, since the process of finding and correctly fixing them is as significant a deliverable as the final metrics.

1. **Short-text language detection weakness** (Stage 3) – discovered by testing realistic short queries after achieving 99.5% benchmark accuracy. Fixed via sentence-initial-biased short-span data augmentation.

2. **Sentiment domain-shift on support queries** (Stage 4) – plain questions misclassified as frustrated. Fixed via train-split-only naturally-phrased augmentation.

3. **Sentiment negation handling** (Stage 4/11) – negated positive statements misclassified as positive with high confidence. Fixed via paired negated-positive/negated-negative augmentation to avoid overcorrection.

4. **Complaint routing bug** (Stage 7) – low-confidence complaint predictions were being downgraded to a generic refusal by the confidence-threshold logic, defeating the requirement that complaints always receive priority handling. Fixed by exempting the `complaint` intent from the confidence-threshold downgrade entirely.

5. **Cold-refusal-to-frustrated-customers bug** (Stage 10) – discovered via live FastAPI testing: an angry, refund-demanding message with an ambiguous specific intent (38% confidence) was routed to a generic refusal rather than escalation. Fixed by making the router **sentiment-aware**: a low-confidence prediction paired with strongly negative sentiment is now treated as an unclear complaint and escalated.

6. **Noise-escalation regression** (Stage 10) – the fix above initially caused pure gibberish input (which had spuriously triggered a confident negative-sentiment label) to be escalated to human review. Fixed by introducing a noise floor below which no prediction is trusted for any purpose, including escalation.

7 Multilingual Testing and Safety

While language detection is genuinely multilingual (trained across 20 languages), sentiment and intent classification are trained exclusively on English-language datasets. This distinction was tested explicitly, both through an automated test suite (`test_multilingual.py`, 27 tests spanning 12 languages) and through live manual testing.

Live testing with a neutral Chinese query (UTF8gbsn?, *"Where is my order?"*) produced a striking result: sentiment was misclassified as 96%-confidence *Negative/Frustrated*, and intent confidence for `billing_and_refunds` was only 24%. Despite this doubly unreliable signal, **the system degraded safely**: the 24% intent confidence fell below the noise floor (30%) established during the Stage 10 bug fix, correctly routing to a polite refusal rather than a false escalation or a hallucinated response. This is presented as a positive finding: a system's value is not only in what it does correctly, but in how safely it fails outside its designed scope.

8 Evaluation Summary

Module	Metric	Result
Language Detection	Test Accuracy	99.53%
Language Detection	Macro F1	0.9953
Sentiment Classification	Test Accuracy	97.70%
Sentiment Classification	Macro F1	0.912
Intent Classification	Test Accuracy	99.78%
Intent Classification	Macro F1	0.9974
RAG Retrieval	Qualitative	4/4 evaluation queries topically correct
Automated Tests	Pass Rate	62/62 (100%)

Table 3: Final evaluation metrics across all system components.

9 Limitations

Consistent with the project's engineering philosophy, limitations are documented explicitly rather than omitted:

- **Language detection**: residual weakness between Spanish/Portuguese and between romanized Hindi/Urdu and Swahili.
- **Sentiment/Emotion dataset mismatch**: `dair-ai/emotion` is Twitter data, not customer-support language; corrected but not perfectly, as documented above.
- **Emotion-to-sentiment mapping**: *surprise* is approximated as *Neutral*, though it can be genuinely positive or negative depending on context.
- **Non-English input to English-only stages**: sentiment and intent are unreliable on non-English text, though the system fails safely rather than acting confidently on bad signal (Section 7).
- **Knowledge base coverage**: RAG responses are limited to the 27 intents represented in the Bitext dataset; questions outside this scope correctly trigger an honest "I don't know" rather than a hallucination, but coverage is inherently bounded.
- **No real backend integration**: the system cannot look up actual order records or process real refunds; it guides customers through process and escalates to human agents.
- **LLM dependency**: generation depends on Groq API availability; failures fall back to a generic retry message.
- **No multi-turn context**: each message is currently processed independently of conversation history.

10 Future Improvements

- Broader adversarial and negation-aware augmentation for sentiment classification.
- Expansion of the RAG knowledge base beyond the Bitext dataset's scope.
- Real order/account system integration for genuine transactional actions.
- Systematic, data-driven threshold tuning for routing confidence.
- Multi-turn conversational context.

11 Conclusion

This project demonstrates a complete, end-to-end NLP system integrating four distinct modeling approaches – classical machine learning, fine-tuned transformers, dense retrieval, and large language model generation – into a single coherent, testable, and honestly-evaluated pipeline. Its most valuable engineering lesson was methodological: strong benchmark metrics on a held-out test set did not guarantee correct real-world behavior, and every meaningful improvement in this project came from deliberately adversarial, manual testing against realistic and edge-case inputs, followed by targeted, well-reasoned fixes and permanent regression tests – a discipline this report has aimed to make as visible as the final numbers themselves.